

Assignment 1: Haskell Basics and Simple Evaluators

CSC324H1, Fall 2025

Due Date: Sept 19, 2025 (11:59 pm)

Overview

In this assignment, you will:

- Practice programming using recursion and datatypes;
- Implement a small-step operational semantics evaluator for the untyped arithmetic expressions;
- Implement a big-step operational semantics evaluator for the untyped arithmetic expressions.

Instructions

A starter project is provided to you. You should complete the functions as required later in the handout and marked with `error "TODO"` in the code. Run the tests provided to check your work. Note that your code will be graded based on additional hidden tests.

Please note that for all assignments:

- It is recommended to only submit all required files as specified on MarkUs.
- You are responsible for making sure that your code has no syntax errors or compile errors. If your file cannot be imported by another file, you may receive a grade of zero.
- If you're not intending to complete one of the functions, do *not* remove the function signature. If you are including a partial solution, make sure it doesn't cause a compile error. If your partial solution causes compile errors, it's better to comment out your solution (but not the signature).
- Do not modify the `module ... where` and `import ...` lines of your code. These lines are crucial for your code to pass the tests.

Part 1: Basic Recursion

(20 pts)

In this part of the assignment, you will be working in the file `RecursionAndDatatypes.hs`. You are *not allowed* to use built-in higher-order functions.

(a) merge

(7 pts)

Implement the function `merge :: [Int] -> [Int] -> [Int]`. The function takes two sorted lists and returns the sorted union of the two lists. Duplicated elements are kept.

In other words, you are implementing the *merge* operation of merge-sort.

(b) takePositive

(13 pts)

Implement the function `takePositive :: Int -> [Int] -> [Int]`.

The function takes a number `n :: Int` and a list `xs :: [Int]`, and returns the first `n` positive elements (strictly > 0) of `xs` in their order of appearance. If there are $< n$ positive elements, then return all the positive elements in their order of appearance.

We encourage you to implement it in a *tail recursive* way (no points will be deducted if you do not do so.)

Part 2: Datatypes and Pattern Matching

(20 pts)

In this part of the assignment, you will be working in the file `RecursionAndDatatypes.hs`.

(a) circumference

(7 pts)

We start with the definition of a datatype:

```
data Shape = Square Float | ...
```

You are to implement the function `circumference :: Shape -> Float`, which calculates the circumference C of a `Shape`.

Here are the formulas for the circumference of the shapes:

Shape with dimension	Formula	Note
Square s	$C = 4s$	
Rectangle $l\ w$	$C = 2(l + w)$	
Circle r	$C = 2\pi r$	Use built-in <code>pi</code>
RightTriangle $a\ b$	$C = a + b + \sqrt{a^2 + b^2}$	Use built-in <code>sqrt</code>

(b) invert, sumTree, maxTree

(13 pts)

We start with the definition of a binary tree:

```
data BinaryTree = Node | Branch BinaryTree Int BinaryTree
```

You are to implement the following functions:

- `invert :: BinaryTree -> BinaryTree`: Inverts the binary tree.
- `sumTree :: BinaryTree -> Int`: Find the sum of all integers in the tree.
- `maxTree :: BinaryTree -> Int`: Find the maximum integer in the tree. If the tree is empty, return the minimum `Int`. You may use the built-in value `minBound` as the minimum integer.

Part 3: Evaluators for the Untyped Arithmetic Expressions (60 pts)

In this part of the assignment, you will be implementing two evaluators for the untyped arithmetic expressions. One evaluator will implement the small-step semantics, the other will implement the big-step semantics. The practice you get from this assignment will also be useful for later assignments, where you will be extending your work to handle more language constructs.

(a) Explanation of Starter Code

Language Syntax The syntax of untyped arithmetic expression is defined in `Term.hs`:

```
data Term = ...
```

For big-step semantics, we also provide the syntax for values:

```
data Value = ... -- Values from Lecture 2
```

```
data NumValue = ... -- Numerical values from Lecture 2
```

Auxiliary Definitions We also introduce a few type synonyms and datatypes across the source files:

```
type ErrorMessage = String
```

```
data StepResult
  = StepOk Term
  | StepError ErrorMessage
  deriving (Eq, Show)
```

```
data EvalResult
  = EvalOk Value
  | EvalError ErrorMessage
  deriving (Eq, Show)
```

Name	Note
<code>ErrorMessage</code>	A type synonym for <code>String</code> .
<code>StepResult</code>	Denotes whether a term can step to a new term in the small-step semantics.
<code>EvalResult</code>	Denotes whether a term can be evaluated to a value in the big-step semantics.
<code>StepOk t, EvalOk v</code>	Indicate a successful stepping/evaluation result.
<code>StepError msg, EvalError msg</code>	Indicate failed stepping/evaluation, with appropriate error message <code>msg</code> .

(b) Small-Step Operational Semantics

One-step Evaluation (`oneStep`) The one-step evaluation function is defined in `SmallStep.hs` with the following signature:

```
oneStep :: Term -> StepResult
```

The function takes a `Term` to step, and returns a result of type `StepResult`, which is either a `(StepOk t')` if there exists t' such that $t \rightarrow t'$ holds, or `(StepError msg)` for some error message `msg`.

Multi-step Evaluation (`multiStep`) We have supplied a multi-step evaluation function `multiStep` in `SmallStep.hs` that uses `oneStep`. It has the following type signature:

```
multiStep :: Term -> Term
```

`multiStep` takes a `Term` to be fully evaluated and returns a result of type `Term`. The result should be the normal form the input evaluates to after a chain of one-step evaluation. You should check `multiStep` is correct with respect to your implementation of `oneStep`.

(c) Big-Step Operational Semantics

Big-step Evaluation (`bigStep`) The big-step evaluator is defined in `BigStep.hs` with the following type signature:

```
bigStep :: Term -> EvalResult
```

The function takes a `Term` to be evaluated, and returns a result of type `EvalResult`, which is either a `(EvalOk v)` if there exists v such that $t \Downarrow v$ holds, or `(EvalError msg)` for some error message `msg`.

Your Tasks and Tips

1. (25 pts) Implement the one-step evaluator `oneStep` in `SmallStep.hs`.
 - The one-step evaluator should essentially implement the one-step evaluation relation $t \rightarrow t'$ as defined in Figure 3-1, 3-2 in TAPL.
 - Good error messages are encouraged for debug purpose, while the specific error message *does not* affect the grading, as long as a `StepError` is returned when it should.
 - You might find Chapters 3, 4 of TAPL useful.
2. (10 pts) Verify the multi-step evaluator `multiStep` in `SmallStep.hs`.
 - *Do not* modify the given `multiStep` implementation. You will however be graded on whether `multiStep` is correct with respect to your implementation of `oneStep`.
 - The multi-step evaluator should be a functional representation of the multi-step evaluation relation $t \rightarrow^* u$ as defined by Definition 3.5.9 in Chapter 3 of TAPL, where u is in normal form.
 - You might find Chapters 3, 4 of TAPL useful.
3. (25 pts) Implement the big-step evaluator `bigStep` in `BigStep.hs`.
 - The big-step evaluator should essentially implement the big-step semantics as presented in Week 2 lecture.
 - The specific error message is not graded, as long as a `EvalError` is returned when it should.